



SISTEMAS DISTRIBUIDOS I
(TA050) CURSO ROCA

Trabajo práctico Escalabilidad

Money Laundering Analysis



29 de mayo de 2026

Cristhian David Noriega
109164

Mateo Alvarez
108666

Santiago Tomas Fassio
109463

Índice

1. Introduccion	3
2. Alcance y casos de uso	3
3. Análisis del conjunto de datos	4
3.1. Estructura del dataset	4
3.2. Variantes disponibles	4
3.3. Conversión de divisas	5
3.4. Campos utilizados por consulta	5
4. Procesamiento y flujo de datos	6
4.1. Cadenas de procesamiento por consulta	6
4.1.1. Consulta 1 — Transacciones USD menores a 50	7
4.1.2. Consulta 2 — Máximo monto por banco origen	7
4.1.3. Resultado final	7
4.1.4. Consulta 3 — Anomalías por comparación de promedios	8
4.1.5. Consulta 4 — Detección del patrón <i>scatter-gather</i>	8
4.1.6. Consulta 5 — Conteo de transacciones menores a 1 USD	9
4.2. Flujo de mensajes	10
4.2.1. Ejemplo: Consulta 1	11
4.3. Actividades de los procesos en el sistema	11
4.3.1. Pre-procesamiento - Gateway	12
4.3.2. File Splitter	13
4.3.3. File Ingestor	13
4.3.4. Consulta 3	14
4.3.5. Post-procesamiento — Cierre del cliente	15
4.4. Arquitectura e Infraestructura	15
4.5. Vista de Desarrollo	16
4.5.1. Diagrama de Paquetes	16
4.6. Vista Lógica	17
4.6.1. Diagrama de Clases	17
4.7. Vista Fisica	17
5. Tareas a implementar	18
5.1. División entre integrantes	18

1. Introduccion

En el presente trabajo se desea desarrollar un sistema distribuido para el analisis y deteccion de anomalias en transacciones entre cuentas bancarias. Para ello se provee un conjunto de consultas predefinidas que ayuden al correspondiente analisis.

2. Alcance y casos de uso

El proyecto *Money Laundering Analysis* consiste en un sistema distribuido que analiza el extracto de transacciones realizadas entre cuentas bancarias en busca de anomalías asociadas a patrones de lavado de activos. El sistema recibe como entrada un conjunto de datos de transacciones bancarias y produce, para cada consulta definida, un reporte con los resultados correspondientes.

El sistema soporta múltiples clientes concurrentes. Cada cliente envía su propio conjunto de transacciones y recibe de forma independiente los resultados de las cinco consultas. Las ejecuciones de distintos clientes son aisladas entre sí: un cliente no tiene acceso a los datos ni a los resultados de otro. No se impone ninguna restricción sobre el subconjunto de registros que cada cliente elija enviar.

Las consultas que el sistema debe responder para cada cliente son las siguientes:

Las consultas que el sistema debe responder son las siguientes:

- **Consulta 1:** Obtener cuenta de origen, cuenta de destino y monto de transacciones en USD con monto menor a 50.
- **Consulta 2:** Obtener nombre de banco, cuenta de origen y monto de la transacción USD máxima por banco.
- **Consulta 3:** Obtener cuenta de origen y monto de transacciones USD en el período [2022-09-06, 2022-09-15] cuyo monto sea menor a un centésimo del promedio del mismo formato de pago en el período [2022-09-01, 2022-09-05].
- **Consulta 4:** Detectar cuentas que presenten el patrón *scatter-gather* con una única cuenta de separación, considerando cuentas que hayan realizado transferencias USD hacia al menos 5 cuentas distintas en el período [2022-09-01, 2022-09-05].
- **Consulta 5:** Contar la cantidad de transacciones del período [2022-09-01, 2022-09-05] con formato de pago Wire o ACH cuyo monto convertido a USD sea menor a 1.

A continuación se presenta el diagrama de casos de uso del sistema, donde se ilustran las interacciones entre el cliente y cada una de las consultas disponibles.

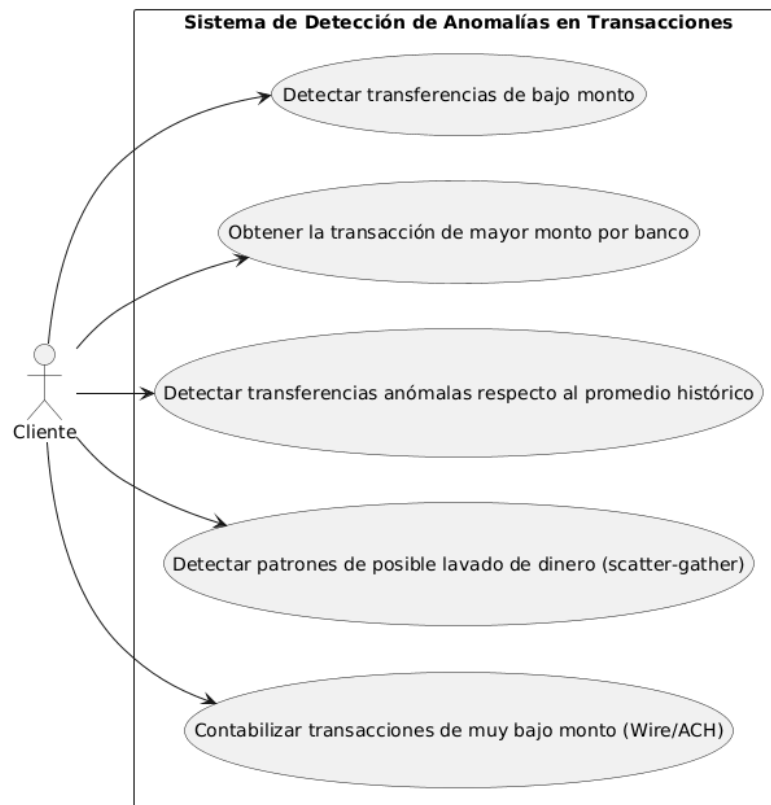


Figura 1: Diagrama de casos de uso, mostrando las interacciones entre el cliente y el sistema.

3. Análisis del conjunto de datos

El sistema trabaja sobre el dataset sintético de transacciones bancarias publicado por IBM para detección de lavado de activos, disponible en Kaggle¹. Este dataset fue generado mediante un simulador que modela el ciclo completo del lavado de activos: *placement*, *layering* e *integration*, y cada transacción se encuentra etiquetada como legítima o sospechosa.

3.1. Estructura del dataset

El dataset se provee en formato CSV. Cada fila representa una transacción entre dos cuentas bancarias y contiene las siguientes columnas:

3.2. Variantes disponibles

El dataset se distribuye en seis variantes combinando dos grupos (*HI* con mayor tasa de lavado y *LI* con menor tasa) y tres tamaños. La siguiente tabla resume sus características principales:

Dado que las consultas 3 y 4 requieren datos hasta el 15 de septiembre de 2022, las variantes *Small* no son suficientes. El sistema está diseñado para operar sobre las variantes *Medium* o *Large*, siendo *HI-Medium* y *LI-Medium* las utilizadas como referencia para validar los resultados contra el notebook patrón provisto por la cátedra.

¹<https://www.kaggle.com/datasets/ealtman2019/ibm-transactions-for-anti-money-laundering-aml>

Campo	Descripción
Timestamp	Fecha y hora de la transacción
From Bank	Banco de origen
Account (origen)	Cuenta de origen
To Bank	Banco de destino
Account (destino)	Cuenta de destino
Amount Paid	Monto enviado
Payment Currency	Moneda de la transacción
Amount Received	Monto recibido
Receiving Currency	Moneda recibida
Payment Format	Formato de pago (Wire, ACH, etc.)
Is Laundering	Etiqueta: 0 legítima, 1 sospechosa

Cuadro 1: Columnas del dataset de transacciones

3.3. Conversión de divisas

El dataset contiene transacciones en múltiples monedas. Las consultas 1, 2, 3 y 4 operan únicamente sobre transacciones cuya moneda original es USD, por lo que en esos pipelines no se realiza conversión de divisas: las transacciones en otras monedas son descartadas por los filtros correspondientes.

La Consulta 5, en cambio, requiere evaluar montos convertidos a USD. Para esto, el sistema cuenta con un `rate.server` que centraliza las tasas de conversión. Durante su inicialización, este servicio intenta cargar las tasas desde una cache local. Si la cache no existe, consulta la API pública de Frankfurter (<https://api.frankfurter.app>) para el período histórico del dataset y persiste el resultado localmente.

Sobre esas tasas se aplica además una tabla de referencia para Q5, alineada con el notebook provisto por la cátedra, que completa o sobrescribe valores necesarios para la consulta, incluyendo Bitcoin. De esta forma, las tasas se tratan como datos de referencia inmutables durante toda la ejecución.

3.4. Campos utilizados por consulta

No todos los campos del dataset son relevantes para todas las consultas. La siguiente tabla detalla qué campos consume cada una:

Consulta	Campos utilizados
Q1	Account (origen), Account (destino), Amount Paid, Payment Currency
Q2	From Bank, Account (origen), Amount Paid, Payment Currency
Q3	Account (origen), Amount Paid, Payment Currency, Payment Format, Timestamp
Q4	Account (origen), Account (destino), Amount Paid, Payment Currency, Timestamp
Q5	Amount Paid, Payment Currency, Payment Format, Timestamp

Cuadro 2: Campos del dataset utilizados por cada consulta

4. Procesamiento y flujo de datos

En esta sección se describe cómo la información atraviesa el sistema desde su ingesta hasta la producción del reporte final. Se presentan primero las cadenas de procesamiento individuales de cada consulta como grafos acíclicos dirigidos (DAG), para luego mostrar el flujo completo del sistema integrando todas las consultas en paralelo.

4.1. Cadenas de procesamiento por consulta

Antes de ramificarse en los pipelines individuales, todas las transacciones atraviesan un filtro común que descarta aquellas cuya moneda no sea USD. Este filtro inicial reduce el volumen de datos de forma temprana, beneficiando a todas las consultas que operan exclusivamente sobre transacciones en dólares. A partir de este punto, los datos fluyen en paralelo por cinco pipelines independientes, uno por consulta. A continuación se describe el DAG del sistema.

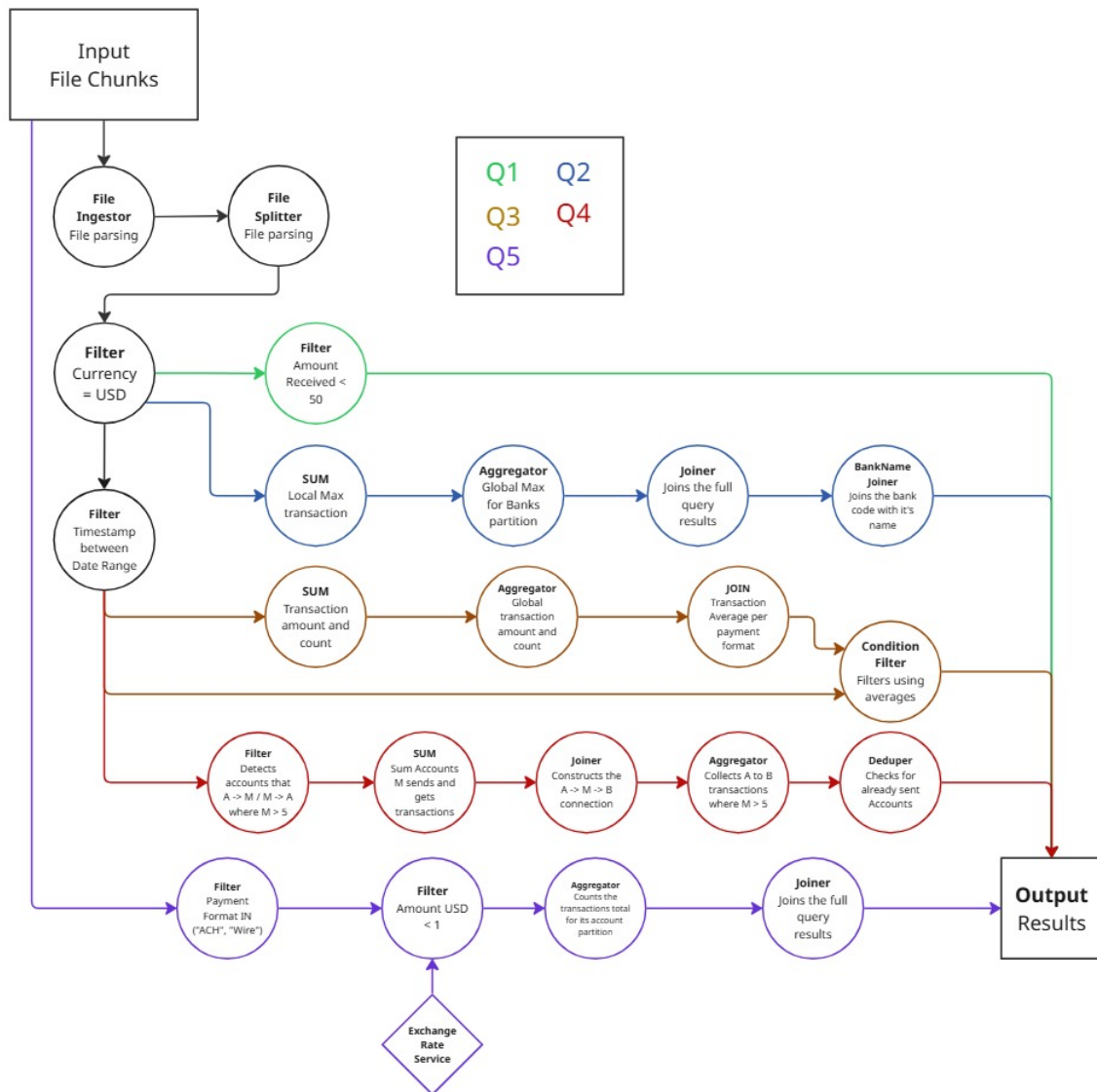


Figura 2: DAG general del sistema con los cinco pipelines en paralelo

4.1.1. Consulta 1 — Transacciones USD menores a 50

La Consulta 1 es el flujo más directo del sistema. Luego del filtro común por moneda, las transacciones en USD pasan por un filtro de monto que conserva únicamente aquellas con `Amount Paid < 50`. Como el resultado de la consulta es un conjunto de registros individuales, no se requiere ninguna etapa de agregación.

Las transacciones aceptadas se envían al Gateway en batches, manteniendo la semántica de un resultado por transacción pero reduciendo la cantidad de mensajes intercambiados por RabbitMQ.

4.1.2. Consulta 2 — Máximo monto por banco origen

La Consulta 2 obtiene, para cada banco origen, la transacción USD de mayor monto y luego la enriquece con el nombre del banco a partir del archivo de cuentas.

El cálculo pesado se distribuye en varias etapas. Primero, `sum_q2` calcula máximos locales por banco. Luego, `aggregation_q2` recibe los parciales ruteados por `bank_id`, de forma que todos los candidatos de un mismo banco terminan en la misma partición. Finalmente, `join_q2` consolida los resultados de todos los aggregators y produce un máximo final por banco.

Para coincidir con la semántica del notebook, se replica el comportamiento de `idxmax`: si dos transacciones de un mismo banco tienen el mismo monto máximo, se conserva la primera aparición en el archivo original. Para eso, las transacciones transportan su número de fila y el desempate se realiza por menor `row_number`.

El enriquecimiento con nombres de banco se realiza en una etapa separada, `q2_bank_name_joiner`. Esta etapa consume dos streams independientes: por un lado, los máximos finales emitidos por `join_q2`; por otro, los batches del archivo de cuentas emitidos por el `file_splitter`. Para cada cliente, mantiene en memoria dos estructuras:

$$q2_results[bank_id] = \text{máximo final del banco}$$
$$bank_names[bank_id] = \text{nombres asociados al banco}$$

El worker no emite resultados hasta recibir el EOF de ambos streams. Recién cuando conoce que recibió todos los máximos y todos los batches de cuentas para un cliente, realiza el join por `bank_id`, agrega el `Bank Name` y envía los resultados al Gateway.

Esta última etapa no está particionada horizontalmente: existe una única instancia de `q2_bank_name_joiner`. La decisión se tomó porque el volumen que recibe ya fue reducido significativamente. No procesa todas las transacciones, sino un máximo por banco más el catálogo de cuentas. Por lo tanto, el costo esperado está asociado a la cantidad de bancos y cuentas, no a la cantidad total de transacciones.

La parte escalable de Q2 es la reducción transaccional previa: `filter_usd`, `sum_q2` y `aggregation_q2`. Si el dominio tuviera una cantidad mucho mayor de bancos o si el catálogo de cuentas creciera al punto de volver esta etapa un cuello de botella, el siguiente paso natural sería particionar también el `q2_bank_name_joiner` por `bank_id`, enviando tanto los máximos como las filas de cuentas al mismo shard.

4.1.3. Resultado final

El resultado final de Q2 contiene, para cada banco origen, la cuenta que realizó la transacción USD de mayor monto, el nombre del banco y el monto correspondiente. La salida coincide con la semántica del notebook, incluyendo el desempate por primera aparición en el archivo original.

4.1.4. Consulta 3 — Anomalías por comparación de promedios

La Consulta 3 se divide en dos ramas. La primera calcula, para el período base, el promedio de monto por formato de pago. Esta rama usa `sum_q3`, `aggregation_q3` y `join_q3`, particionando por `payment_format` para que cada promedio pueda calcularse de forma distribuida.

La segunda rama contiene las transacciones candidatas del período posterior. Como estas transacciones pueden llegar antes que los promedios, el `q3_barrier` actúa como punto de sincronización: espera tanto los promedios como los candidatos antes de emitir resultados.

Para evitar mantener todos los candidatos en memoria, el barrier los persiste temporalmente en disco y luego los recorre de forma incremental. Cuando dispone de los promedios, filtra las transacciones cuyo monto es menor al 1 % del promedio de su formato de pago y envía las anomalías al Gateway.

4.1.5. Consulta 4 — Detección del patrón *scatter-gather*

Las transacciones USD del rango temporal de Q4 ingresan al pipeline de *Scatter-Gather*, compuesto por cinco etapas distribuidas y escalables horizontalmente. Cada etapa recibe su partición según una clave de hash distinta y propaga el EOF downstream con la cantidad de registros enviados a cada destinatario. Esto permite que la siguiente etapa sepa cuándo dispone de toda la información correspondiente a un cliente.

q4_filter — prefiltro por cuenta de origen. Cada instancia recibe su partición por hash de la cuenta origen A . Para cada A , mantiene un conjunto de cuentas destino distintas con tope en seis y bufferiza sus transferencias en memoria. Cuando A alcanza seis destinos distintos, es decir $|\text{destinos}(A)| > 5$, queda calificada: sus transferencias bufferizadas se reenvían río abajo y las posteriores fluyen directamente. Las cuentas que nunca califican se descartan al cierre del cliente.

Por cada transferencia calificada $A \rightarrow M$ se emiten dos registros `Q4CountedEdge`, ambos ruteados por el intermediario M : un `IN`, que indica que M recibe de A , y un `OUT`, que indica que A envía a M .

q4_sum — acumulador por intermediario. Cada instancia recibe los *counted edges* cuyo intermediario M le corresponde por hash, y acumula:

$$\text{incoming}[M][A] += \text{count}$$

$$\text{outgoing}[M][B] += \text{count}$$

Al cierre del cliente, planifica cada intermediario M . Si la cardinalidad

$$|\text{incoming}[M]| \times |\text{outgoing}[M]|$$

es chica, todo M se procesa como un único bloque. Si M es un *hub*, se subdivide en una grilla de:

$$16 \times 16 = 256$$

bloques mediante:

$$a_bucket = \text{hash}(A) \bmod 16$$

$$b_bucket = \text{hash}(B) \bmod 16$$

Por cada cuenta A en su fila y cada cuenta B en su columna, se emiten registros `Q4BlockJoinEdge` dirigidos al bloque correspondiente:

$$(M, a_bucket, b_bucket)$$

q4_joiner — join por bloque. Cada instancia es dueña de un conjunto de bloques (M, a_bucket, b_bucket) . Mientras llegan los *half-edges*, los acumula en diccionarios locales separados por bloque y por rol, ya sea IN u OUT.

Al recibir EOF de todos los `q4_sum` upstream, recorre cada bloque y realiza el producto cartesiano local. Para cada par (A, B) con $A \neq B$, calcula:

$$path_count = count(A \rightarrow M) \times count(M \rightarrow B)$$

Luego topea dicho valor a 6 y emite:

$$Q4PairPaths(A, B, path_count)$$

ruteado por hash del par (A, B) . La división en bloques permite distribuir el peor caso, por ejemplo un intermediario *hub* con miles de contrapartes, entre múltiples joiners en lugar de concentrar toda la carga en una única instancia.

q4_aggregator — agregador por par. Cada instancia es dueña de los pares (A, B) que le corresponden por hash. Cada `Q4PairPaths` recibido representa la contribución de un único intermediario M al conteo total de caminos de ese par. El agregador acumula:

$$total[(A, B)] += path_count$$

Cuando el total supera 5, el par califica. En ese momento, se emiten las dos cuentas A y B como candidatos `Q4AccountId`, ruteados por cuenta, y se descarta el contador asociado al par. Los pares que nunca cruzan el umbral quedan en memoria y se liberan al cierre del cliente.

q4_deduper — deduplicación por cuenta. Una misma cuenta puede aparecer en múltiples pares calificados. Cada instancia de `q4_deduper` es dueña de las cuentas que le corresponden por hash y las mantiene en un conjunto, absorbiendo silenciosamente los repetidos.

Al cierre del cliente, las cuentas únicas se envían al Gateway en batches de `Q4AccountId`. Como el Gateway espera un único EOF por queue de resultado, los dedupers no líderes reportan su conteo emitido al `q4_deduper_0`; sólo el líder envía el EOF final de Q4 una vez que todas las réplicas le reportaron.

El resultado final de Q4 es el conjunto de cuentas únicas $(Bank, Account)$ que participan en algún patrón *scatter-gather* calificado. Dicho resultado se entrega al cliente a través del Gateway usando la misma interfaz que las demás queries.

4.1.6. Consulta 5 — Conteo de transacciones menores a 1 USD

La Consulta 5 cuenta las transacciones que cumplen tres condiciones: el formato de pago debe ser *Wire* o *ACH*, la fecha debe pertenecer al rango indicado por el enunciado y el monto convertido a dólares debe ser menor a 1 USD.

El pipeline comienza en `filter_q5_format`, que recibe las transacciones desde el fanout inicial y descarta todos los formatos que no pertenecen a la consulta. Las transacciones válidas se envían en batches hacia la siguiente etapa, reduciendo la cantidad de mensajes físicos en RabbitMQ sin cambiar la semántica lógica del flujo.

Luego, `filter_q5_usd` realiza la conversión de moneda. Para esto obtiene las tasas desde el `rate_server` mediante RPC y evalúa cada transacción según:

$$amount_usd = amount \times rate(currency, date)$$

Si la transacción está dentro del rango temporal de Q5 y cumple:

$$amount_usd < 1$$

se reenvía hacia los aggregators de Q5.

La etapa `aggregation_q5` realiza conteos parciales por cliente. Cada instancia acumula cuántas transacciones válidas recibió y, al llegar el EOF correspondiente, emite un resultado parcial:

$$count_i = \#\{tx : amount_usd < 1\}$$

Finalmente, `join_q5` recibe los parciales de todos los aggregators y calcula el conteo total por cliente:

$$count = \sum_i count_i$$

Una vez recibidos todos los EOF esperados, emite una única fila de resultado hacia el Gateway.

El resultado final de Q5 es un conteo por cliente con la cantidad total de transacciones **Wire** o **ACH** cuyo monto convertido a dólares es menor a 1 USD.

4.2. Flujo de mensajes

En esta sección se detalla la comunicación entre los componentes del sistema para cada consulta, mostrando el intercambio de mensajes desde que el cliente envía las transacciones hasta que recibe el resultado. Todos los componentes intermedios realizan coordinación interna ante la llegada de un EOF. Esta coordinación interna se encarga de verificar el correcto procesamiento de todos los datos enviados por el cliente o la etapa anterior y la propagación de la señal de fin de transmisión (*EOF*) hacia el siguiente stage, garantizando que ningún componente emita su resultado parcial antes de haber procesado la totalidad de los mensajes que le corresponden.

4.2.1. Ejemplo: Consulta 1

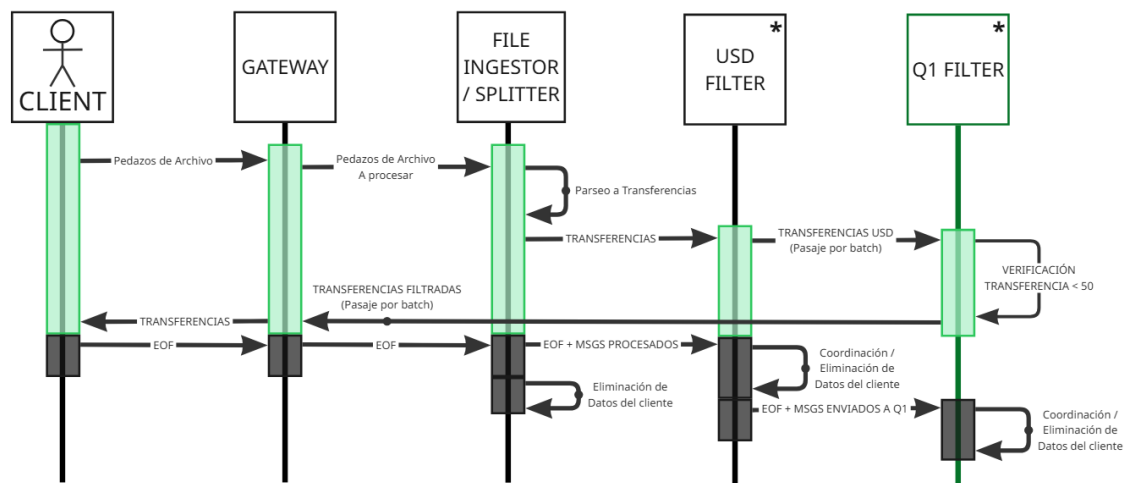


Figura 3: Diagrama de secuencia — Consulta 1

El cliente se comunica con el *File Splitter* y el *File Ingestor*, los cuales traducen los pedazos de archivo enviados por el cliente a transacciones. Estas transacciones pasan a manos del *USD Filter*. Este componente descarta las transacciones en moneda distinta a USD y pasa el resto al *Q1 Filter*, que aplica el criterio de monto menor a 50 y devuelve las transacciones que lo cumplen directamente al Gateway para su entrega al cliente. Al finalizar el envío de datos, el cliente emite un *EOF* que se propaga en cascada: el gateway propaga a los procesadores de archivos, los cuales propagan el EOF al filtro USD y a su vez al Filtro de Q1. Cada mensaje de EOF contiene el dato de cuántos mensajes procesó.

Cuando una instancia de un worker recibe un mensaje de EOF, este se vuelve el líder. Esta instancia líder envía un mensaje al resto de instancias pidiendo los mensajes procesados. Cada Instancia devuelve un mensaje al líder con la cantidad de mensajes procesados, y el líder hace la adición de todas las instancias. Si la suma es igual a la esperada, se hace una orden de flush, donde las instancias envían los datos correspondientes a la siguiente etapa (esto en caso de workers como SUM o Aggregator. Como filter forwarda sin necesidad de EOF, no hace flush), y también eliminan los datos correspondientes al cliente en cuestión.

Finalmente el líder, al recibir que todas las instancias flushearon los datos, se encarga de mandar un EOF a la siguiente etapa conteniendo los mensajes forwardados por cada instancia.

4.3. Actividades de los procesos en el sistema

En la siguiente sección se describe las actividades realizadas por los actores del sistema, tomando tres etapas: el pre-procesamiento, cómo el Gateway gestiona la concurrencia entre clientes, el procesamiento donde muestran la coordinación interna de las queries más complejas (3 y 4), y el post-procesamiento que muestra cómo convergen los resultados y finaliza el ciclo de vida del cliente.

4.3.1. Pre-procesamiento - Gateway

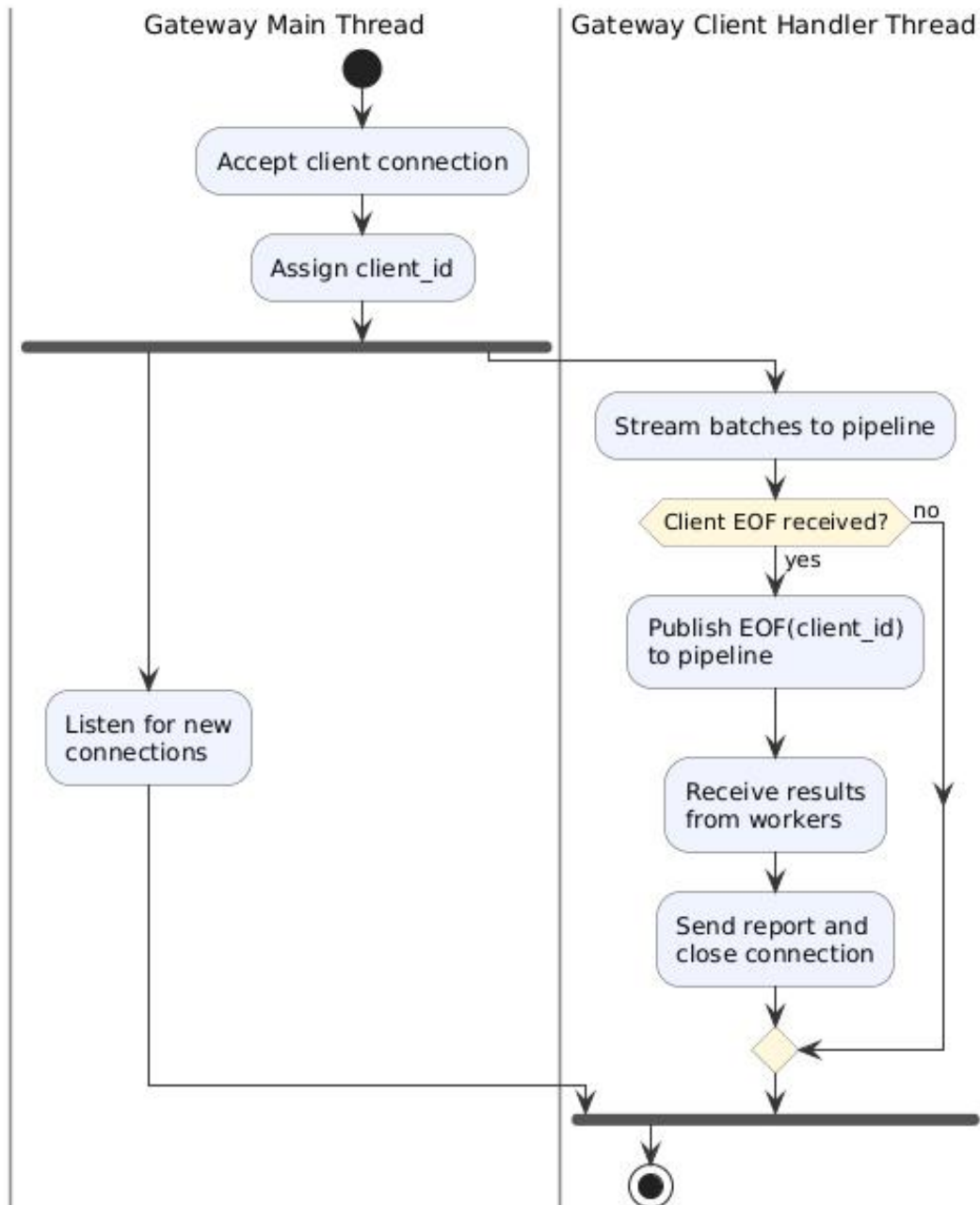


Figura 4: Diagrama de Actividades

El Gateway es el único punto de entrada del sistema. Su responsabilidad es aceptar conexiones de clientes, asignar un `client_id` único a cada sesión y crear un thread dedicado para manejar el ciclo de vida de ese cliente. El thread principal no procesa los datos del cliente: luego de aceptar la conexión vuelve inmediatamente a escuchar nuevas conexiones, permitiendo atender múltiples clientes concurrentes.

Cada *Client Handler Thread* recibe del cliente los archivos de entrada en forma de chunks de

bytes. El Gateway no parsea CSV ni aplica lógica de negocio; simplemente encapsula esos chunks con metadata de protocolo (`client_id`, tipo de archivo, offset, tamaño, etc.) y los publica hacia el `file_splitter`. De esta manera, el streaming desde el cliente hacia el sistema distribuido se mantiene desde el primer componente.

Cuando el cliente termina de enviar sus archivos, el Gateway propaga la señal de fin de entrada para ese `client_id`. A partir de ese momento, el cierre del procesamiento queda coordinado por los EOF internos del pipeline: `file_splitter`, `file_ingestor` y los workers de cada consulta van propagando sus propios EOF con contadores lógicos.

En paralelo, el Gateway consume los resultados producidos por las consultas y los reenvía al cliente. La conexión no se cierra al recibir el primer resultado ni al finalizar una única consulta, sino cuando llegaron los EOF terminales de todas las consultas activas para ese cliente. Recién entonces el Gateway envía la señal final al cliente y cierra la sesión.

4.3.2. File Splitter

Inmediatamente después del Gateway, el nodo File Splitter es el responsable de transformar los chunks binarios provenientes del cliente en lotes de líneas (line batches) consumibles por el resto del pipeline. Para cada archivo en curso, identificado por la dupla (`client id`, ruta relativa), mantiene un buffer que acumula bytes hasta detectar un fin de línea, momento en el cual cada línea completa se incorpora al lote actual. Cuando el lote alcanza el tamaño máximo configurado se publica en la cola correspondiente, separando el flujo de transacciones del flujo de cuentas. Al recibir el EOF del cliente, vacía cualquier línea pendiente, publica el lote remanente y propaga un EOF downstream informando la cantidad total de líneas de transacción emitidas, valor que más tarde permitirá al File Ingestor validar que recibió todos los datos. Las instancias son intercambiables: los chunks se reparten por hash entre las réplicas, de modo que puede escalarse la cantidad de splitters sin acoplamiento con el resto del sistema.

4.3.3. File Ingestor

El nodo `file_ingestor` consume los lotes producidos por el `file_splitter` y se encarga del parseo a estructuras tipadas. Por cada línea del lote construye un objeto `Transaction` con los campos correspondientes: banco origen, cuenta origen, banco destino, cuenta destino, monto, moneda, formato de pago, timestamp y número de fila original, entre otros.

Los objetos resultantes se publican en batches sobre un exchange fanout de transacciones. A este exchange se suscriben los filtros iniciales de los pipelines, permitiendo que cada consulta reciba el flujo completo y decida qué registros conservar.

La señal de fin de transmisión requiere coordinación entre réplicas. Cada `file_ingestor` procesa una parte de los batches y reporta cuántas transacciones publicó para cada cliente. Cuando se completan todos los aportes, se emite un único EOF descendente con el total lógico de transacciones publicadas. De este modo, los filtros downstream observan una señal de cierre consistente independientemente de la cantidad de réplicas de ingestor activas.

4.3.4. Consulta 3

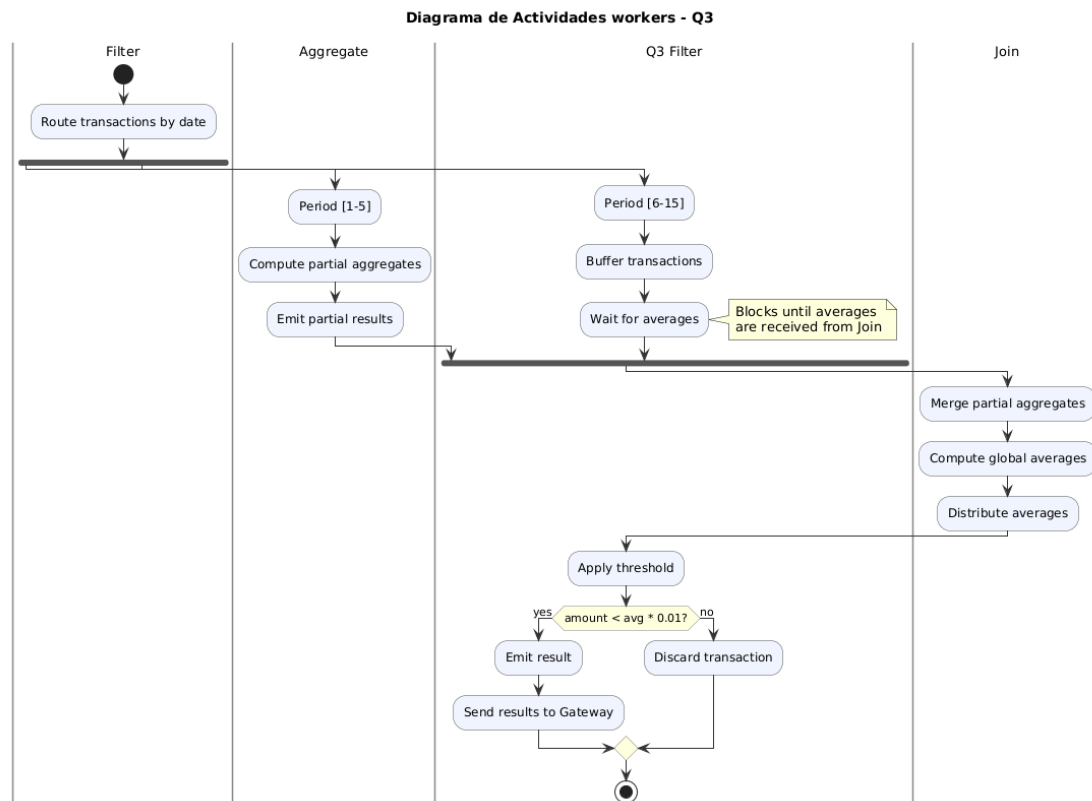


Figura 5: Diagrama de Actividades

La Consulta 3 fue elegida porque es la que tiene la coordinación más complicada: hay dos ramas que trabajan por separado pero que en algún momento sí o sí tienen que sincronizarse para poder dar un resultado.

Una decisión de diseño importante fue usar un fork, lo que implica que los cuatro workers arrancan totalmente independientes, sin un orquestador central que los esté coordinando. La sincronización se termina resolviendo de forma “natural” a través del middleware: el Q3 Filter queda esperando en el estado de “Wait for averages”, pero no porque alguien lo bloquee explícitamente, sino simplemente porque todavía no tiene mensajes para consumir hasta que el Join publique los promedios. Este es el único caso en todo el sistema donde un worker queda esperando a otro, y pasa justamente por la dependencia entre los dos períodos de la consulta.

En esa misma línea, se decidió bufferear en el Q3 Filter en lugar de procesar en streaming. La razón es se debe a que sin conocer el threshold, no hay forma de decidir qué transacciones se deberían emitir. Ese buffer crece de forma lineal con el volumen del período [6–15], y es el principal riesgo de memoria de esta consulta. Si el dataset fuera más grande que la RAM disponible, la alternativa sería volcar a disco con un merge externo.

Por último, la sincronización antes del Join refleja que ese nodo recién puede emitir resultados cuando recibió todos los agregados parciales de las réplicas del SUM. Esta “barrera” asegura que el promedio calculado sea realmente global, y no un promedio parcial sobre solo una parte de los datos.

4.3.5. Post-procesamiento — Cierre del cliente

El cierre de una sesión de cliente no depende de que una cola quede vacía ni de un timeout, sino de la recepción explícita de los EOF de las consultas activas.

Cada pipeline terminal envía sus resultados al Gateway y, al finalizar, emite un mensaje EOF asociado al `client_id`. Ese EOF incluye la cantidad de resultados lógicos emitidos por la etapa, permitiendo que el Gateway valide que recibió el stream completo de esa consulta.

El Gateway mantiene, para cada cliente, el conjunto de consultas pendientes de finalización. A medida que recibe los EOF terminales, marca cada consulta como completa. Mientras todavía queden consultas pendientes, continúa recibiendo y reenviando resultados al cliente.

Cuando todas las consultas activas de un cliente fueron completadas, el Gateway envía la señal de fin de resultados al cliente y cierra la conexión asociada a esa sesión. De esta forma, el cierre del cliente queda coordinado por mensajes explícitos del protocolo y no por observación del estado de RabbitMQ.

4.4. Arquitectura e Infraestructura

En la siguiente sección se muestra el diagrama de robustez, en el cual figura la interacción del cliente con nuestro sistema, y como dentro del mismo interactúan los diferentes actores, como escalan y flujo de datos entre ellos.

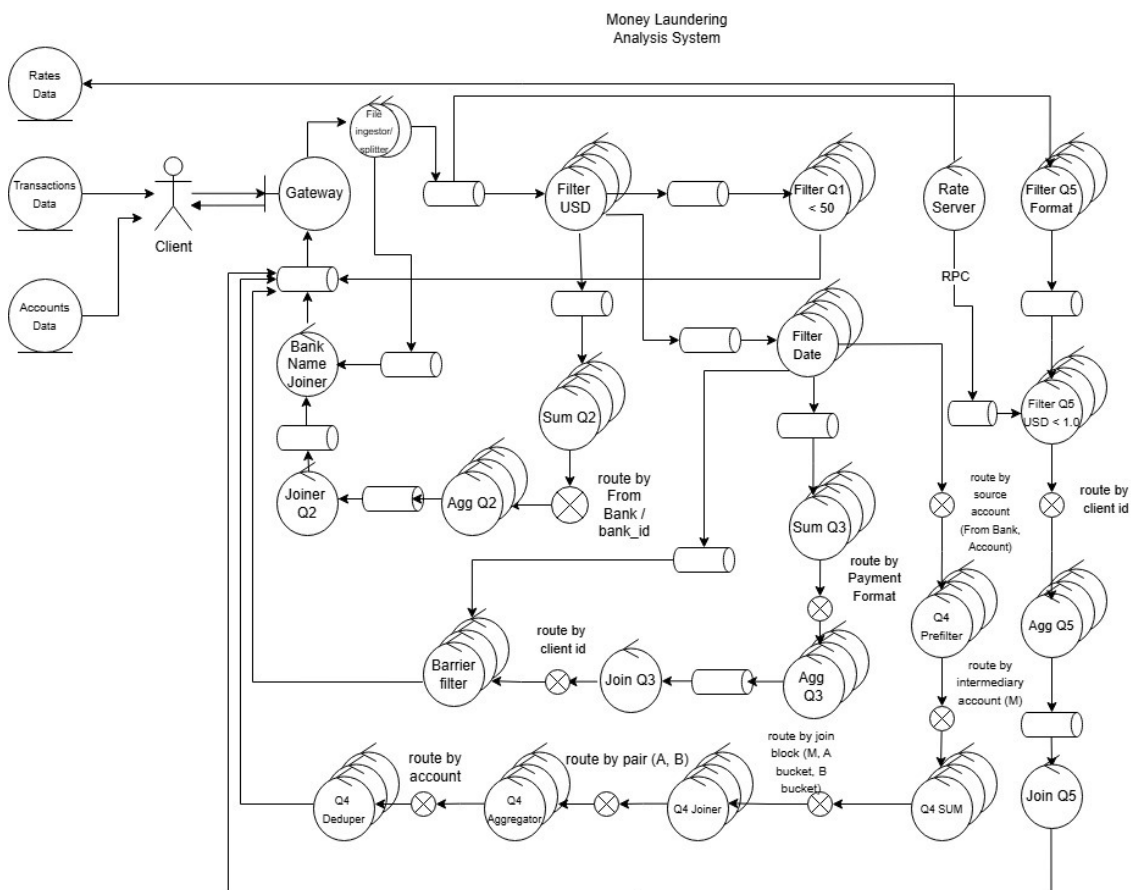


Figura 6: Diagrama de Robustez

En el diagrama se observa el flujo e interacción de las 5 queries, y como se hace uso de colas

de RabbitMQ para la comunicacion entre los diferentes workers/controladores. Un detalle a aclarar es la notacion de exchange, que denota el ruteo por hash a los diferentes controladores del step siguiente en el pipeline. Esto esta relacionado al nivel de escalamiento de nuestro sistema, el cual escala en direccion horizontal aumentando las replicas de los controladores en cada caso en particular.

4.5. Vista de Desarrollo

4.5.1. Diagrama de Paquetes

Los paquetes que se destacan:

- **Applications:** Contiene principalmente el cliente, que se comunica con el Gateway para enviar los datos de las transacciones a los workers correspondientes.
- **Messages:** Define los mensajes que se pueden utilizar para la comunicación entre Workers, y los datos enviados devuelta al cliente. Además contiene un módulo dedicado a serializar y deserializar grandes batches de mensajes.
- **Domain:** Define las estructuras internas de los datos.
- **Workers:** Son los encargados de procesar y transformar los datos correctamente según las consignas. Se divide en File Splitter, File Ingestor, Filters, Sums, Joins, Aggregators, entre otros según corresponda. Reciben datos del paquete de aplicación.
- **External Services:** Se encarga de tener los datos correctos sobre la conversión de unidad de cambio. Se comunica con los workers correspondientes de Q5
- **RabbitMQ:** Encargado de definir las estructuras de comunicación entre los componentes. Todos los paquetes que necesiten comunicarse usan tanto exchanges o colas definidas en este paquete.

Cabe aclarar que en el diagrama se muestra el paquete EOFController el cual esta como propuesta de libreria de manejo del protocolo EOF y que cada worker la pueda utilizar.

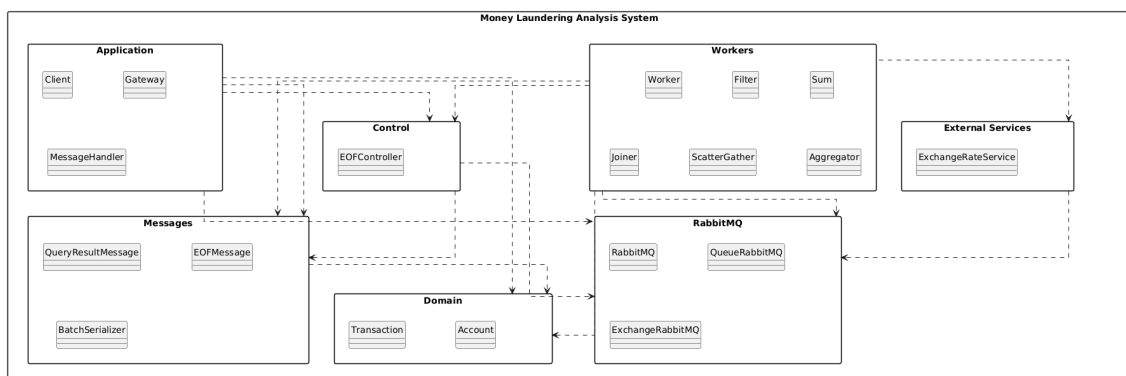


Figura 7: Diagrama de Paquetes

4.6. Vista Lógica

4.6.1. Diagrama de Clases

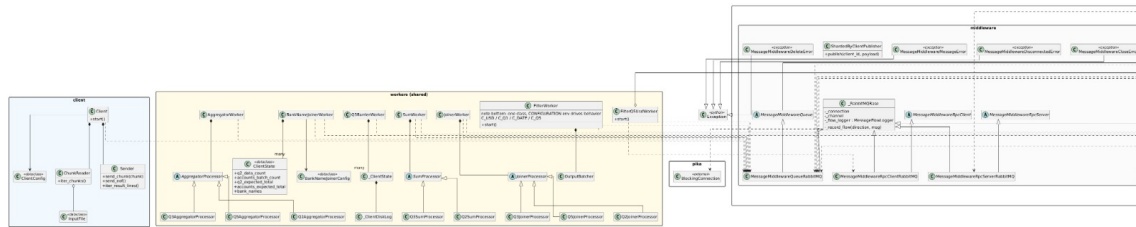


Figura 8: Diagrama de Clases

El diagrama de clases presenta las clases principales del sistema y sus relaciones. Se aclara que este diagrama nos permite visualizar las responsabilidades centrales de cada componente, pero no contiene los detalles completos de implementación.

En la capa de aplicación, el *Client* se ocupa de iniciar la comunicación con el servidor y enviarle al *Gateway* los dos archivos que contienen todas sus transacciones a analizar y sus cuentas de banco. Una vez terminado el pipeline en el lado del servidor, recibe los resultados y los almacena. Por otro lado, el *Gateway* actúa como punto único de entrada: acepta clientes, administra sus identificadores, recibe los resultados y envía las respuestas finales. El *File Splitter* y *File Ingestor* se encargan de construir batches de transacciones y de publicarlos en las colas correspondientes al inicio del pipeline.

La comunicación se encapsula en el paquete *RabbitMQ*. La clase *RabbitMQ* define operaciones comunes de envío, consumo, confirmación y cierre de consumo, mientras que *QueueRabbitMQ* y *ExchangeRabbitMQ* especializan esta lógica para comunicación por colas o por exchanges. La serialización de batches, queda concentrada en *BatchSerializer*.

El paquete *Workers* contiene la clase *Worker*, que reúne el comportamiento común de las etapas de procesamiento: consumo de batches, manejo de EOF, liberación de resultados acumulados y limpieza del estado de un cliente. Las clases *Filter*, *Sum*, *Joiner*, *ScatterGather* y *Aggregator* representan las operaciones lógicas necesarias para resolver las consultas. Cada worker mantiene diccionarios internos por cliente, permitiendo procesar múltiples clientes concurrentemente sin mezclar datos ni resultados.

Finalmente, el paquete *Domain* contiene las entidades del problema: *Transaction* y *Account*. Estas modelan las transferencias bancarias y las cuentas involucradas, y son utilizadas por los distintos workers durante el procesamiento, para resolver las cinco queries.

4.7. Vista Física

En la siguiente sección figura la infraestructura de despliegue para el funcionamiento del sistema distribuido. Como mencionamos anteriormente, en el diagrama de robustez, los nodos/workers escalan horizontalmente, es decir aumentando más replicas de los mismos. Esto se ve en el siguiente diagrama, junto con los demás nodos involucrados:

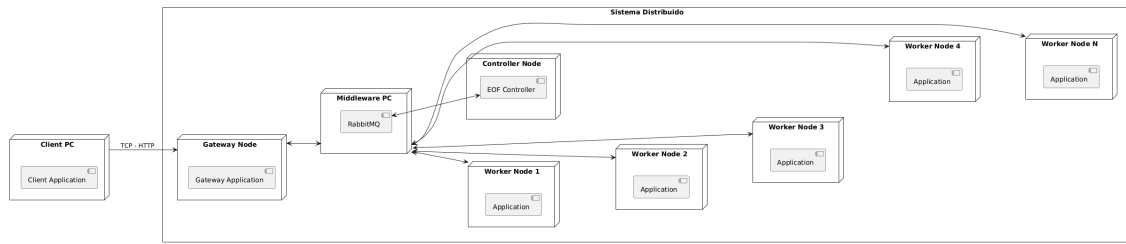


Figura 9: Diagrama de Despliegue

El diagrama de despliegue muestra la infraestructura física sobre la que corre el sistema distribuido. Se pueden identificar cuatro tipos de nodos:

- **Client PC:** ejecuta la aplicación cliente, que se conecta al Gateway Node mediante TCP/IP para enviar las transacciones y recibir los resultados de las cinco consultas.
- **Gateway Node:** punto de entrada único al sistema. Ejecuta la aplicación Gateway, que recibe los datos del cliente, los distribuye al pipeline de procesamiento y recolecta los resultados para devolverlos al cliente.
- **Middleware PC:** Tiene la instancia de RabbitMQ que actúa como broker de mensajería central para toda la comunicación entre workers. Todos los nodos del sistema se conectan a este broker para publicar o consumir mensajes.
- **Worker Nodes (1...N):** cada nodo ejecuta una instancia de la aplicación worker, que puede corresponder a cualquiera de los workers del pipeline. El sistema escala horizontalmente agregando más Worker Nodes, lo que permite aumentar el número de réplicas de cualquier etapa del pipeline.

5. Tareas a implementar

5.1. División entre integrantes

Tarea	Santiago Fassio	Mateo Alvarez	Cristhian Noriega
Estructura del proyecto y módulos		✓	
Docker-compose, Makefile y configuración	✓	✓	✓
Wrapper RabbitMQ y middleware	✓	✓	✓
Protocolo de mensajes y serialización			✓
Filters	✓		
Sums		✓	
Aggregators			✓
Joiners			✓
Scatter Gathers		✓	
Client Endpoint		✓	
Gateway		✓	
Exchange Rate Service			✓
Q1 Filter	✓		
Q2 SUM, Aggregator y Join			✓
Q3 SUM, JOIN y Filter	✓		
Q4 Mapper, Linker y Detector		✓	
Q5 Filter, Joiner y Aggregator			✓
EOF - Controller	✓		
Funcionalidad de escalabilidad			✓
Pruebas de integración (make test)	✓		
Graceful quit / SIGTERM		✓	

Cuadro 3: División de tareas entre integrantes